homework6

2026-10-03

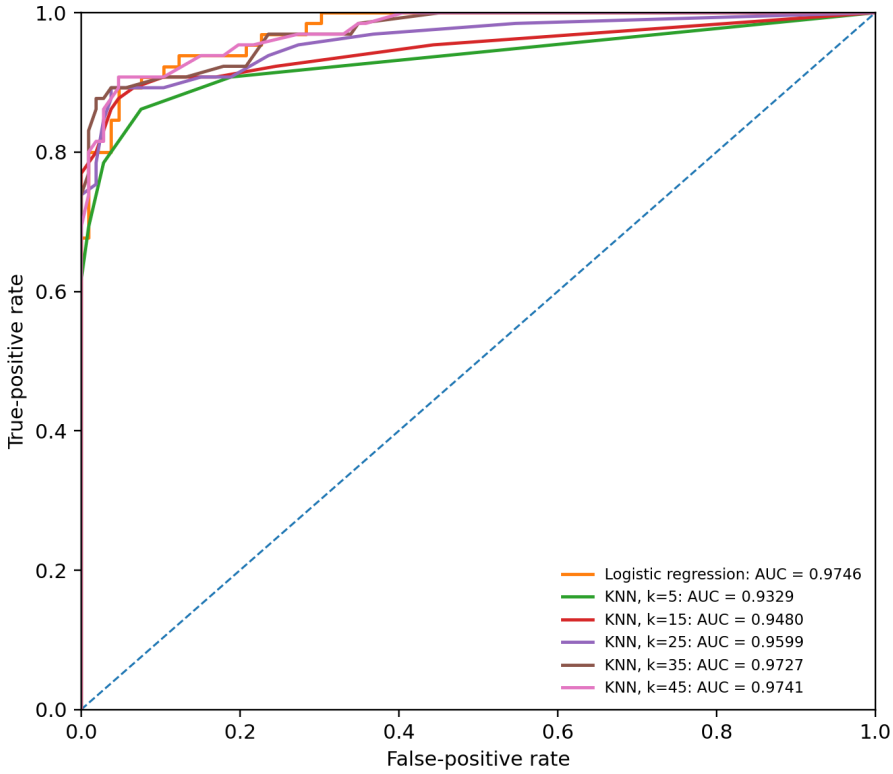
Homework 6

Question 1

Seed used: 432. The supplied split gives 398 training observations and 171 test observations. All predictor means and standard deviations were estimated from the training data only.

Model	Test AUC
Logistic regression	0.9746
KNN, $k = 5$	0.9329
KNN, $k = 15$	0.9480
KNN, $k = 25$	0.9599
KNN, $k = 35$	0.9727
KNN, $k = 45$	0.9741

I would choose logistic regression because it has the largest test AUC, 0.9746. KNN with $k = 45$ is almost tied, but logistic regression gives the best ranking on this test split with a simpler fitted model.



```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt

predictors = ["mean_radius", "mean_texture", "mean_smoothness"]
mu = train[predictors].mean()
sd = train[predictors].std(ddof=1)
X_train = ((train[predictors] - mu) / sd).to_numpy()
X_test  = ((test[predictors] - mu) / sd).to_numpy()
y_train = train["y"].to_numpy()
y_test  = test["y"].to_numpy()

logit = LogisticRegression(penalty=None, solver="lbfgs", max_iter=1000)
logit.fit(X_train, y_train)

k_values = [5, 15, 25, 35, 45]
names = ["Logistic regression"] + [f"KNN, k={k}" for k in k_values]
probs = [logit.predict_proba(X_test)[:, 1]]

for k in k_values:
    fit = KNeighborsClassifier(n_neighbors=k, weights="uniform",
                               metric="euclidean", algorithm="brute")
    fit.fit(X_train, y_train)
    probs.append(fit.predict_proba(X_test)[:, 1])

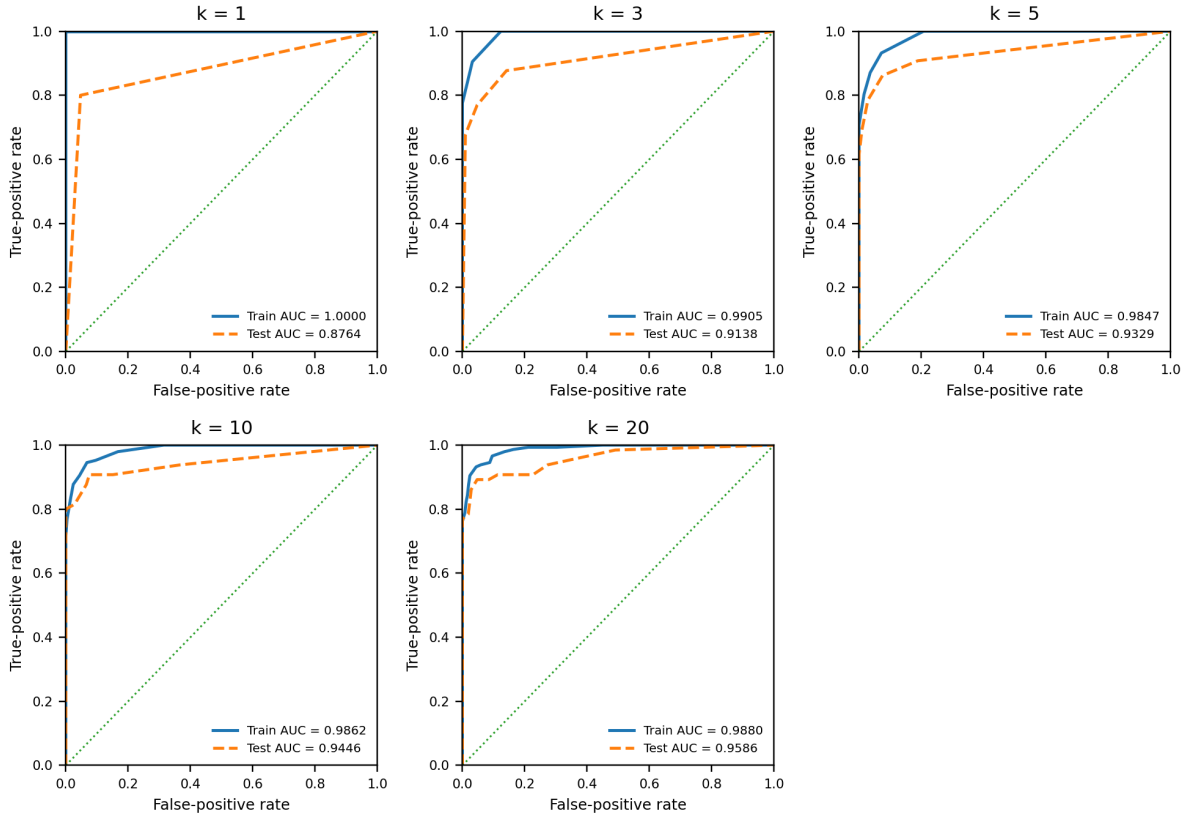
auc_values = [roc_auc_score(y_test, p) for p in probs]
print(pd.DataFrame({"Model": names, "Test AUC": auc_values}))

for name, p, auc in zip(names, probs, auc_values):
    fpr, tpr, _ = roc_curve(y_test, p)
    plt.plot(fpr, tpr, label=f"{name}: AUC={auc:.4f}")
plt.plot([0, 1], [0, 1], "--")
plt.xlabel("False-positive rate")
plt.ylabel("True-positive rate")
plt.legend()
plt.show()
```

Question 2

k	Training AUC	Test AUC
1	1.0000	0.8764
3	0.9905	0.9138
5	0.9847	0.9329
10	0.9862	0.9446
20	0.9880	0.9586

At $k = 1$, every training observation is its own nearest neighbor, so the training AUC is exactly 1 even though the test AUC is only 0.8764. As k increases, the training-test gap becomes much smaller because the model is less dependent on individual training observations, so training performance better reflects performance on new data.




```
k_values = [1, 3, 5, 10, 20]
rows = []
fig, axes = plt.subplots(2, 3, figsize=(10, 7))

for i, k in enumerate(k_values):
    fit = KNeighborsClassifier(n_neighbors=k, weights="uniform",
                               metric="euclidean", algorithm="brute")
    fit.fit(X_train, y_train)

    p_train = fit.predict_proba(X_train)[:, 1]  # includes itself
    p_test = fit.predict_proba(X_test)[:, 1]
```

```
    auc_train = roc_auc_score(y_train, p_train)
    auc_test = roc_auc_score(y_test, p_test)
    rows.append([k, auc_train, auc_test])

    ax = axes.flat[i]
    fpr, tpr, _ = roc_curve(y_train, p_train)
    ax.plot(fpr, tpr, label=f"Train AUC={auc_train:.4f}")
    fpr, tpr, _ = roc_curve(y_test, p_test)
    ax.plot(fpr, tpr, "--", label=f"Test AUC={auc_test:.4f}")
    ax.plot([0, 1], [0, 1], ":")
    ax.set_title(f"k = {k}")
    ax.set_xlabel("False-positive rate")
    ax.set_ylabel("True-positive rate")
    ax.legend()

axes.flat[-1].axis("off")
plt.tight_layout()
plt.show()
print(pd.DataFrame(rows, columns=["k", "Training AUC", "Test AUC"]))
```

Question 3

Selected report: `homework-05.pdf`, Question 2.

Mistake / improvement: The lasso cross-validation standardizes the predictors once using the full training set before the folds are evaluated. The assignment requires the means and scales to be recomputed using only the training portion of each fold.

Exact quote:

```
Xtr_mean <- colMeans(Xtr)
Xtr_sd <- apply(Xtr, 2, sd) * sqrt((n_train - 1) / n_train)
Xtr_c <- scale(Xtr, center = Xtr_mean, scale = Xtr_sd)
ytr_c <- ytr - ytr_mean
cvfit <- cv.glmnet(Xtr_c, ytr_c, alpha = 1, lambda = lambdas,
                   nfolds = 10, standardize = FALSE, intercept = FALSE)
```

Assessment: This lets each validation fold influence the preprocessing used to evaluate it, so the cross-validation step does not exactly follow the required procedure. The fold-specific mean, scale, and response mean should be estimated from that fold's training rows only.

Correction: Perform the preprocessing inside the fold loop, select the penalty from those validation errors, and then refit once on the complete training set.

```
set.seed(100)
foldid <- sample(rep(1:10, length.out = n_train))
cv_mse <- matrix(NA, 10, length(lambdas))

for (v in 1:10) {
  tr <- foldid != v
  va <- !tr

  mu <- colMeans(Xtr[tr, ])
  sc <- sqrt(colMeans(sweep(Xtr[tr, ], 2, mu, "-")^2))
  ybar <- mean(ytr[tr])

  X_tr <- sweep(sweep(Xtr[tr, ], 2, mu, "-"), 2, sc, "/")
  X_va <- sweep(sweep(Xtr[va, ], 2, mu, "-"), 2, sc, "/")

  fit <- glmnet(X_tr, ytr[tr] - ybar, alpha = 1,
                lambda = lambdas, standardize = FALSE,
                intercept = FALSE)
  pred <- predict(fit, newx = X_va, s = lambdas) + ybar
  cv_mse[v, ] <- colMeans((pred - ytr[va])^2)
}

best_lambda <- lambdas[which.min(colMeans(cv_mse))]
```

After selecting `best_lambda`, the model should be refit on all 400 training observations using means and scales calculated from all 400 training rows, and the test MSE and number of nonzero slopes should be reported again.

GitHub Issue link: `https://github.com/nupurkalele1/stat432-fall2026/issues/2`